

Big Mart Sales Prediction

A Machine Learning Approach to Retail Sales Forecasting

Data & Machine Learning Engineering Report

Prepared by

Ahasna

Machine Learning Group Project

Team: Ahasna · KaveeN · Prasadi

1. Introduction to the Dataset

The dataset used for this project is the Big Mart Sales Prediction dataset, sourced from Kaggle (kaggle.com/datasets/shivan118/big-mart-sales-prediction-datasets). It contains 8,523 records collected across ten Big Mart outlets, describing individual products along with details about the store each was sold in. The goal is to predict `Item_Outlet_Sales` — the amount a given product sold for at a given outlet — which makes this a regression problem rather than a classification one.

Each row in the dataset is described by eleven input features and the target variable, giving twelve columns in total. Of these, four are numeric, one is an integer, and the remaining seven are text-based categorical fields. The table below summarises what each feature represents.

Feature	Description
<code>Item_Identifier</code>	Unique identifier for each product
<code>Item_Weight</code>	Weight of the product
<code>Item_Fat_Content</code>	Whether the product is Low Fat or Regular
<code>Item_Visibility</code>	Percentage of total display area given to the product in-store
<code>Item_Type</code>	The category the product belongs to (e.g. Dairy, Meat, Household)
<code>Item_MRP</code>	Maximum Retail Price of the product
<code>Outlet_Identifier</code>	Unique identifier for each store
<code>Outlet_Establishment_Year</code>	The year the outlet first opened
<code>Outlet_Size</code>	The size of the outlet — Small, Medium, or High
<code>Outlet_Location_Type</code>	The tier of area the outlet is located in
<code>Outlet_Type</code>	Whether the outlet is a grocery store or a type of supermarket
<code>Item_Outlet_Sales</code> (target)	The sales amount of the product at that outlet — what the model predicts

Before any modelling could begin, the dataset needed a closer look. Running an initial exploration showed that two columns had missing values: `Item_Weight` was missing in 1,463 rows, and `Outlet_Size` was missing in 2,410 rows. No other column had missing entries, and a check for duplicate rows returned none.

One data quality issue stood out immediately. The `Item_Fat_Content` column was meant to hold only two categories, but a value count revealed five distinct labels — 'Low Fat', 'Regular', 'LF', 'reg', and 'low fat' — where 'LF' and 'low fat' were really just inconsistent spellings of 'Low Fat', and 'reg' was shorthand for 'Regular'. This kind of inconsistency is common in real-world data and had to be cleaned before the column could be encoded correctly.

2. Feature Engineering

Six feature engineering techniques were applied to the dataset, each addressing a specific issue found during exploration. The reasoning behind each one is explained below rather than simply listing what was done, since understanding why a technique was needed matters as much as applying it.

2.1 Handling Missing Values

`Item_Weight` is a continuous numeric field, so its missing values were filled with the column mean. This keeps the overall distribution of weights intact without introducing bias toward any particular product type. `Outlet_Size`, on the other hand, is categorical, so its missing values were filled with the mode — the most frequently occurring size — which is the conventional approach when the mean or median doesn't apply.

2.2 Fixing and Encoding Categorical Variables

With the inconsistent `Item_Fat_Content` labels identified, the first step was to standardise them down to two clean categories, Low Fat and Regular. After that, every categorical column — `Item_Fat_Content`, `Item_Type`, `Outlet_Size`, `Outlet_Location_Type`, and `Outlet_Type` — was converted into numeric form using Label Encoding. Machine Learning algorithms operate on numbers, not text, so this step was necessary before any model could be trained.

2.3 Feature Scaling

The three numeric columns — `Item_Weight`, `Item_Visibility`, and `Item_MRP` — sit on very different scales; `MRP` values run into the hundreds while visibility is a small fraction. Left unscaled, a model could end up giving undue weight to whichever feature happens to have larger raw numbers. Applying `StandardScaler` puts all three on a comparable scale, so the model judges them by their actual predictive value rather than their magnitude.

2.4 Creating a New Feature — `Outlet_Age`

Rather than feeding the raw `Outlet_Establishment_Year` into the model, a new feature called `Outlet_Age` was derived by subtracting the establishment year from 2013, the year the dataset was collected. An outlet's age is a more directly meaningful signal than the calendar year it opened — older, more established stores tend to sell differently from newer ones — so this transformation gives the model something more useful to learn from.

2.5 Outlier Treatment

`Item_Visibility` contained a number of unusually large values that could distort the model's predictions if left as they were. These were identified using the IQR (Interquartile Range) method and capped at the upper limit ($Q3 + 1.5 \times \text{IQR}$) instead of being removed outright. Capping was chosen over deletion so the full size of the dataset could be preserved while still limiting the influence of extreme values.

2.6 Binning (Optional Enhancement)

As an optional addition, `Item_MRP` can be grouped into price bands — Low, Medium, High, and Very High — using equal-width binning. This wasn't essential to the final model but was left in the notebook as a straightforward way to add a categorical view of pricing alongside the continuous `MRP` value.

3. Model Training and Evaluation

Once the data was cleaned and encoded, it was split into training and testing sets using an 80/20 split, with a fixed random seed so the split is reproducible. Two regression models were then trained on the same data — a Linear Regression model as a baseline, and a Random Forest Regressor, which is generally better suited to capturing the kind of non-linear patterns retail sales data tends to have.

Both models were evaluated on the held-out test set using two standard regression metrics: RMSE (Root Mean Squared Error), which measures the average size of prediction errors in the same units as sales, and R^2 Score, which measures how much of the variation in sales the model is able to explain.

Model	RMSE	R^2 Score
Linear Regression	1137.42	0.5240
Random Forest Regressor (selected)	1086.36	0.5658

The Random Forest Regressor outperformed Linear Regression on both metrics — a lower RMSE, meaning its predictions were closer to the actual sales figures on average, and a higher R^2 , meaning it explained noticeably more of the variance in sales (56.6% compared to 52.4%). This gap makes sense: relationships such as how `Item_MRP` and `Outlet_Type` jointly affect sales are rarely linear, and Random Forest is far better equipped to model that kind of interaction than a straight-line model. For this reason, the Random Forest Regressor was selected as the final model.

4. Saving the Model for Deployment

Once the Random Forest model was confirmed as the better performer, it was saved to disk using `joblib` as `model.joblib`, so it could later be loaded by the backend without needing to be retrained. Alongside the model, the fitted `StandardScaler` and the five Label Encoders were also saved, as `scaler.joblib` and `encoder.joblib` respectively. This step matters more than it might seem — the backend has to transform any new input data in exactly the same

way this notebook did before making a prediction, so passing along the exact scaler and encoders, rather than re-fitting new ones, is what keeps predictions consistent.

These three files were handed off to the backend team for integration into the application's /predict API endpoint, completing the Data and Machine Learning Engineering portion of this project.